Codexion — A Beginner's Guide to Threads, Locks and Scheduling

> A learning guide for 42 students starting **Codexion** (subject v1.1). It explains the concepts so you can design and write **your own** implementation. It is **not** a solution: you won't find a coder routine, a monitor routine or a project heap here.
>
> **Source of truth:** the subject PDF, v1.1. Requirements below come from that document; design suggestions, interpretations, and test assumptions are labeled separately. Use this guide alongside the subject, man pages, small experiments, and peer discussion. Examples illustrate possible designs; they are not a required implementation plan. Claims marked ⚠️ **verify in man page** are ones you should check yourself (`man 3 <function>`, or `man 3p <function>` for the POSIX version).
>
> *This guide was written with AI assistance. Following the subject's AI rules, test and check everything in it before you rely on it.*

Table of Contents

1. What Codexion is (and how it differs from Philosophers)
2. Threads 101
    - 2.1 A familiar example first
    - 2.2 The analogy used in this guide
    - 2.3 Program vs. process vs. thread
    - 2.4 CPU, cores and what "running" really means
    - 2.5 The OS scheduler
    - 2.6 Concurrency vs. parallelism
    - 2.7 How a Codexion coder actually spends its time
    - 2.8 What the subject tells you about the threads
    - 2.9 Why concurrency is hard
3. The pthread API
    - 3.1 Thread lifecycle
    - 3.2 Mutexes
    - 3.3 Condition variables
4. Time in C
5. Concurrency problems
6. Scheduling: FIFO vs. EDF
7. Priority queues and binary min-heaps
8. Architecture overview
9. Logging
10. Testing and debugging
11. Common mistakes checklist
12. README requirements
13. Further resources

---

1. What Codexion is (and how it differs from Philosophers)

N coders sit around a circular table with a Quantum Compiler in the middle. There is one USB dongle between each pair of neighbours, so there are **N dongles**. To **compile**, a coder must hold **both** of their dongles, the left one and the right one. After completing a compile they put the dongles back, **debug**, then **refactor**, and then try to compile again. Burnout still applies during every phase: if compiling itself takes longer than the burnout interval, holding the dongles does not keep the coder alive.

In the subject's words:

> *"Compiling quantum code requires two dongles plugged in simultaneously, one in each hand: a coder takes their left and right dongles to compile."*
>
> *"If a coder did not start compiling within time_to_burnout milliseconds since the beginning of their last compile or the beginning of the simulation, they burn out."*
>
> *"After being released, a dongle is unavailable until its cooldown has passed."*

Note the word **start**: burnout is measured from the *start* of the last compile, not its end. The simulation ends at the first burnout, or once every coder has compiled at least `number_of_compiles_required` times.

```
./codexion number_of_coders time_to_burnout time_to_compile time_to_debug \
           time_to_refactor number_of_compiles_required dongle_cooldown scheduler
```

If you have done Philosophers, much of this will be familiar:

Philosophers	**Codexion**
philosopher	coder
fork	USB dongle
eat / sleep / think	compile / debug / refactor
die	burn out
5th argument optional	**all 8 arguments mandatory**
whoever grabs the fork first gets it	**a scheduler (`fifo` or `edf`) decides who gets it**
fork is free right after release	**dongle has a cooldown** after release
monitor design is up to you	**a separate monitor thread is required**
—	**you must implement a priority queue (heap)**
—	condition variables are allowed (`pthread_cond_*`)

Other points from the subject worth knowing early: - Global variables are forbidden. - Submit the Makefile and C source/header files in `coders/`; the program is named `codexion`. - Use C and follow the 42 Norm. The small examples here explain concepts; check their integration and formatting against the Norm yourself. - No libft. - Compile with `cc` and include `-Wall -Wextra -Werror -pthread`. Provide `$(NAME)`, `all`, `clean`, `fclean`, and `re` Makefile rules, without unnecessary relinking.

---

2. Threads 101

> *"Concurrency and synchronization have become vital skills: knowing how to let many developers or processes work without stepping on each other's toes is as important as writing the code itself."* — Codexion subject, Introduction

2.1 A familiar example first

Right now your laptop is probably running a browser, a music player, a terminal and an editor, all "at the same time". Your CPU has maybe 4 or 8 cores, yet the system is juggling **hundreds** of threads. Try it:

```
nproc                # available logical CPUs (Linux/coreutils)
ps -eLf | wc -l      # roughly how many threads exist right now
```

So how can 8 cores run 1,000 threads? Most threads are **waiting** most of the time (for a key press, a network packet, a timer...), and the operating system **switches** between the ready ones very quickly. The switching is so fast that everything *looks* simultaneous.

Codexion is exactly this situation, just smaller: a handful of coder threads, mostly waiting (on dongles or timers), sharing a few CPU cores.

2.2 The analogy used in this guide

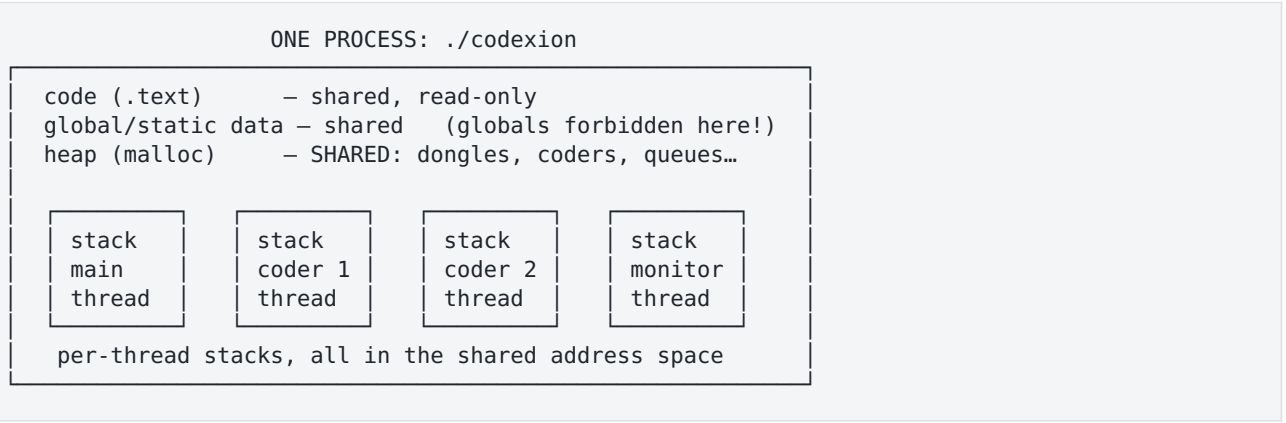
Let's turn the Codexion hub into a picture of your computer:

Computer concept	Hub analogy
Program (`codexion` on disk)	The *plan* for running a hub, written on paper. It does nothing by itself
Process (`./codexion …` running)	A real hub building, opened from that plan. It has its own rooms, whiteboards and locked doors. Other buildings can't walk in
Thread	A coder working inside the building
Shared memory (heap, global/static data)	The building's whiteboards and tables: every coder can see and change them
Stack	Each coder's notebook; others can access it if given a pointer
CPU core	A desk with a working computer. Only a coder **sitting at a desk** actually does work
OS scheduler	The hub manager who decides who sits at which desk and for how long
Context switch	One coder stands up and packs their notebook, and the next one sits down and unpacks theirs
Blocked thread	A coder waiting in the hallway for a dongle. They don't need a desk while they wait
Mutex	A key. Only the holder may touch what's behind the door

Keep this table in mind. Every later section uses it.

2.3 Program vs. process vs. thread

- A **program** is a file: machine code plus data, sitting on disk.
- A **process** is a running instance of a program. The OS gives it its own **address space** (its own view of memory), its own open files, and at least one thread.
- A **thread** is one *flow of execution* inside a process: a current position in the code (instruction pointer), registers, and a stack. A process starts with one thread (the one running `main`), and `pthread_create` adds more.



	Process	**Thread**
Memory	Its own private address space	**Shares** the address space with the other threads of its process
Created with	`fork()` (not allowed here)	`pthread_create()`
Cost to create and switch	Higher (new address space)	Lower (same address space)
Communication	Pipes, files, signals…	Plain variables in shared memory
Has its own	Address space and process resources	Execution state: **stack**, registers, thread ID; also thread-specific attributes such as a signal mask
On an unhandled fatal fault	Usually terminates that process	Usually terminates **the whole process**, including its other threads

The subject chooses threads, not processes:

> *"Each coder must be represented by a thread (using pthread_create)."*

Watch out: If `main` returns (or anyone calls `exit`), the **whole process ends**, and every thread is killed mid-work. That's why `main` must `pthread_join` all threads before it returns.

Stack vs. heap in practice: An automatic local variable belongs to the invocation of the function on that thread, but another thread can access it if you share its address. Its lifetime must then cover those accesses, and concurrent reads and writes need synchronization. The same rules apply to heap objects: what matters is who can access the object, not where it was allocated.

2.4 CPU, cores and what "running" really means

For a first model, imagine one running software thread per **logical CPU**. A physical core may expose more than one logical CPU through simultaneous multithreading (SMT, including Intel Hyper-Threading); those hardware threads share execution resources. `nproc` reports available logical CPUs, which may differ from the number of physical cores.

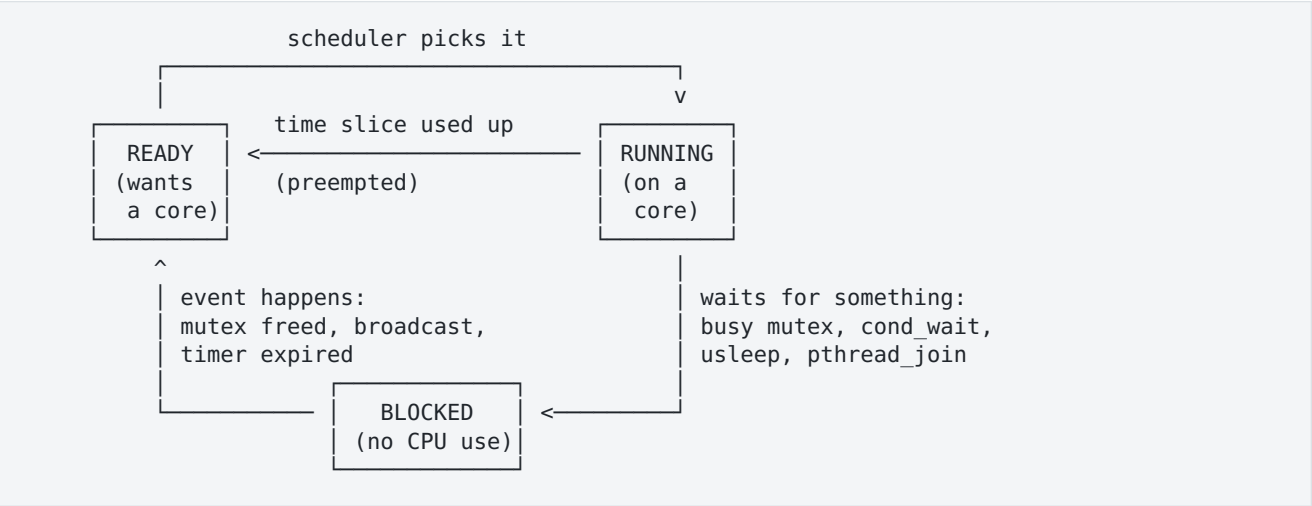
```
          CPU (4 cores)
┌──────────┬──────────┬──────────┬──────────┐
│ core 0   │ core 1   │ core 2   │ core 3   │     <- 4 desks
│ coder 3  │ monitor  │ (idle)   │ firefox  │     <- who sits there right now
└──────────┴──────────┴──────────┴──────────┘

Waiting in the hallway (blocked, no desk needed):
  coder 1 (wants dongle 1)   coder 2 (debugging = sleeping)   coder 4 …
```

Your program doesn't own the CPU. It shares it with every other program on the machine, and during evaluation that may include Valgrind, a test script and a browser.

2.5 The OS scheduler

The **scheduler** is the part of the kernel that decides which thread runs on which core, and for how long. On Linux it schedules **threads**, not processes. A useful simplified model has three states (real operating systems distinguish more):



Key ideas:

- **Time slices and preemption.** A running thread gets a short time slice, typically a few milliseconds (the exact value varies). When it runs out, the scheduler can **preempt** the thread: pause it anywhere, even in the middle of `counter++`, and run someone else. *You never control where your thread gets interrupted.*
- **Context switch.** Swapping one thread for another means saving its registers and loading the next one's. It's fast (on the order of microseconds) but not free.
- **Blocked threads do not continuously consume CPU.** Sleeping or blocking gives up the logical CPU, but each thread still needs memory and kernel resources. Frequent wakeups and contention also have costs, so measure rather than assuming that any thread count is cheap.
- **No ordering promises.** The scheduler doesn't promise *which* ready thread runs next or *when*. A thread you created first might start last. Your program must be correct for **every** possible order.
- **Waking up takes time.** When your `usleep` expires, the thread becomes *READY*, not *RUNNING*. If all cores are busy, it waits in line. That's one reason sleeps are imprecise (§4).

Do not confuse the **OS scheduler**, which assigns CPU time, with the **Codexion scheduler** (`fifo` / `edf`), which arbitrates dongles. For this assignment, design around the OS scheduler rather than depending on a particular thread execution order.

2.6 Concurrency vs. parallelism

- **Concurrency:** several tasks are *in progress* during the same period. They may take turns on one core.
- **Parallelism:** several tasks run *at the same instant* on different cores.

```
Concurrency on 1 core (taking turns):
core 0: [coder1][coder2][coder1][coder3][coder2][coder1] ...

Parallelism on 3 cores (truly simultaneous):
core 0: [coder1 coder1 coder1 coder1 coder1 ...]
core 1: [coder2 coder2 coder2 coder2 coder2 ...]
core 2: [coder3 coder3 coder3 coder3 coder3 ...]
```

Your threads may experience **both**, depending on the available logical CPUs: some can run in parallel, and their execution can also be interleaved. Bugs can come from either. A correct program doesn't depend on how many cores the machine has.

2.7 How a Codexion coder actually spends its time

The simulation does not need to compile real code. Compiling, debugging and refactoring can be represented by *waiting for time to pass*. Time checks, locking, logging and queue operations still consume CPU, and their cost depends on the implementation and machine.

```
coder 1:  [wait dongles]──[compile: sleep 200ms]──[debug: sleep 200ms]──[refactor: sleep 200ms]──[wait…
CPU use:   ~0              ~0                      ~0                    ~0
           ↑ tiny bursts of real work: lock, check, log, unlock
```

With nonzero phase durations and sensible waits, CPU use should usually be modest. Sustained high CPU is a reason to investigate spinning (§5), excessive polling or logging. It is not proof of a bug: many threads or zero-duration phases can also produce substantial work.

2.8 What the subject tells you about the threads

The subject's rules map directly onto threading concepts:

> *"Coders do not communicate with each other." "Coders do not know if another coder is about to burn out."*

The subject combines these model rules with EDF arbitration, which needs deadlines. A useful separation of responsibilities is to let coder routines request resources, let scheduling logic compare request metadata, and let the monitor detect burnout. Shared locks, queues and notifications provide the coordination; avoid adding a separate neighbour-rescue policy to coder behaviour. The subject does not prescribe a module layout.

> *"A separate monitor thread must detect burnout precisely and stop the simulation."*

Shared state needs protection:

> *"To prevent coders from duplicating dongles, you must protect each dongle's state with a mutex (pthread_mutex_t)."*

"Duplicating" a dongle is exactly the race from the next section: two coders both read "dongle free" and both take it.

2.9 Why concurrency is hard

`counter++` is not a C synchronization operation. Conceptually, it reads a value, adds one, and writes it back; the exact machine instructions depend on the compiler and CPU:

```
1. load  counter from memory into a register
2. add   1 to the register
3. store the register back to memory
```

The scheduler can preempt a thread between any two of them, and on two cores the threads can even run the steps at the same instant:

```
counter = 5
Thread A: load (5)
                        Thread B: load (5)
Thread A: add  -> 6
                        Thread B: add  -> 6
Thread A: store 6
                        Thread B: store 6      <- one increment is lost
```

Replace `counter` with "dongle 2 is free" and you get two coders holding the same dongle.

This is called a **data race**. The rules are: - You can't control when the OS switches threads, so a race may hide for 999 runs and appear on the 1000th, often during evaluation, on a different machine. - In C, a data race is **undefined behavior**. Beyond wrong values, the compiler may assume no other thread changes a variable. It can then cache the value in a register, which turns `while (!stop)` into an infinite loop. - `volatile` does **not** fix this. Mutexes do. Locking and unlocking act as "memory fences" that make writes visible to the next thread that takes the same lock.

The rest of this guide is about controlling who touches shared data, and when.

---

3. The pthread API

Compile and link with `-pthread`. Include `<pthread.h>`.

For the pthread functions used here, check their returned status directly: successful calls return 0 and failures return an error number, rather than reporting it through `errno`. `ETIMEDOUT` from a timed wait is an expected outcome to handle, not necessarily a program failure. Other pthread APIs outside this project have different return conventions.

Allowed functions (from the subject)

```
pthread_create, pthread_join,
pthread_mutex_init, pthread_mutex_lock, pthread_mutex_unlock, pthread_mutex_destroy,
pthread_cond_init, pthread_cond_wait, pthread_cond_timedwait,
pthread_cond_broadcast, pthread_cond_destroy,
gettimeofday, usleep, write, malloc, free, printf, fprintf, strcmp, strlen, atoi, memset
```

> **Not allowed:** `pthread_detach` and `pthread_cond_signal`, so this guide doesn't document them. As a result: - **Every thread you create must be joined.** - **Explicit condition-variable notifications use `pthread_cond_broadcast`; timed waits can also expire.**
>
> Other functions you may know are not allowed either: `clock_gettime`, `nanosleep`, `pthread_mutex_trylock`, `pthread_condattr_setclock` and semaphores.

3.1 Thread lifecycle

`pthread_create`

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start_routine)(void *), void *arg);
```

What it does	Starts a new thread that runs `start_routine(arg)`. The new thread's ID is stored in `*thread`.
Returns	`0`, or an error number: `EAGAIN` (out of resources, or thread limit reached), `EINVAL` (bad `attr`), `EPERM` (no permission for the scheduling settings in `attr`).
In Codexion	One call per coder and one for the monitor. Pass `NULL` for `attr`.
Pitfalls	See the list below.

Common pitfalls: - **Passing `&i` (a loop counter) as `arg`.** All threads get the *same address*, and `i` keeps changing while they start. Pass a pointer to that coder's own struct instead. - **Assuming the new thread runs immediately, or in creation order.** It may not. - **Ignoring the return value.** If creation fails halfway, you still have to stop and join the threads you *already* created before you free anything.

`pthread_join`

```
int pthread_join(pthread_t thread, void **retval);
```

What it does	Blocks until `thread` finishes. If `retval` is not `NULL`, stores the thread's return value there. Releases the thread's resources.
Returns	`0` on success, otherwise an error number. Do not rely on error detection for invalid use: joining yourself, a non-joinable thread, or the same thread concurrently is not portable and can have undefined behaviour.
In Codexion	Main joins every coder and the monitor before it destroys mutexes and frees memory.
Pitfalls	See the list below.

Common pitfalls: - **Joining a thread that is blocked forever** (for example, waiting on a condition variable nobody will broadcast). Your program hangs at exit, so your shutdown must wake everyone up first (§8). - **Joining the same thread twice** is undefined behavior. - **Not joining at all** leaks thread resources. Valgrind may report them, and `pthread_detach` is not allowed.

Minimal example

```
#include <pthread.h>
#include <stdio.h>

static void *say_hello(void *arg)
{
  int *id;

  id = (int *)arg;
  printf("coder %d says hello\n", *id);
  return (NULL);
}

int main(void)
{
  pthread_t threads[3];
  int     ids[3];
  int     i;
  int     created;

  i = 0;
  while (i < 3)
  {
    ids[i] = i + 1;
    if (pthread_create(&threads[i], NULL, say_hello, &ids[i]) != 0)
      break ;
    i++;
  }
  created = i;
  i = 0;
  while (i < created)
    pthread_join(threads[i++], NULL);
  return (created != 3);
}
```

Run it a few times and the order of the lines will change. That is normal: the OS decides who runs when. If creation fails, this demo joins the threads already started before returning failure. It assumes the joins succeed; check return values and design an error path in the project.

3.2 Mutexes

A **mutex** (from "mutual exclusion") is a lock that only one thread can hold at a time. Think of it as the key to a small room: anyone who wants to touch the data inside must hold the key, and everyone else waits at the door.

The subject requires it: *"you must protect each dongle's state with a mutex"*.

`pthread_mutex_init`

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
```

- **What it does:** Initializes the mutex. With `attr = NULL` you get the default mutex type.
- **Returns:** `0`, or e.g. `EAGAIN`, `ENOMEM`, `EPERM`.
- **In Codexion:** Initialize every mutex (dongles, logging, simulation state…) **before** you start any thread.
- **Pitfalls:**
  - Initializing a mutex that is already initialized is undefined behavior.

    - Copying a `pthread_mutex_t` by value (e.g. inside a struct you copy) and then using the copy is also undefined behavior. Always work through pointers.

`pthread_mutex_lock` / `pthread_mutex_unlock`

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

- **What they do:**
    - `lock` blocks until the calling thread owns the mutex.
    - `unlock` releases it, and one of the waiting threads (if any) can then take it.
- **Returns:** `0`, or an error. Errors depend on the mutex type: an *error-checking* mutex returns `EDEADLK` if you lock it twice, or `EPERM` if you unlock a mutex you don't own. For the **default** type, both mistakes are **undefined behavior**: often a silent deadlock or a corrupt state. ⚠️ verify in man page (`man 3p pthread_mutex_lock`)
- **In Codexion:** Protect reads **and** writes of shared mutable state with the same synchronization protocol. For example, the monitor reading `last_compile_start` must synchronize with the coder updating it. Configuration initialized before thread creation and never modified afterwards can be read without repeatedly taking a mutex.
- **Pitfalls:**
    - **Forgetting to unlock on an early `return`.** This is the classic bug. Every `return` between a lock and its unlock needs its own unlock.
    - **Holding a lock while you sleep or print for a long time.** Other threads stall, and with tight timings someone burns out.
    - **Taking two mutexes in different orders in different places** leads to deadlock (§5).

`pthread_mutex_destroy`

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

- **What it does:** Destroys the mutex. It must not be used again until it is re-initialized.
- **Returns:** `0`, or an error number. Destroying a **locked** mutex, or one that a thread is still using (for example in a `pthread_cond_wait`), is **undefined behavior**. Some implementations return `EBUSY`, but don't count on it. ⚠️ verify in man page
- **In Codexion:** Call it only **after all threads are joined**, then free the memory that held the mutex.

Minimal example: fixing the lost increment

```
#include <pthread.h>

typedef struct s_counter
{
  pthread_mutex_t lock;
  long      value;
} t_counter;

static void *increment(void *arg)
{
  t_counter *c;
  int     i;

  c = (t_counter *)arg;
  i = 0;
  while (i < 100000)
  {
    pthread_mutex_lock(&c->lock);
    c->value++;
    pthread_mutex_unlock(&c->lock);
    i++;
  }
  return (NULL);
}
```

Initialize the mutex and set `value = 0`, then start two threads with the same `t_counter *` and join both: with successful calls, the result is 200000. Remove the locks and the program has undefined behaviour; a lower count is one possible symptom, but even 200000 would not make the program correct.

3.3 Condition variables

The problem they solve

A coder wants dongle 3, but it is taken. What should the coder do?

- **Busy-wait:** repeatedly lock, check availability, and unlock without sleeping. This wastes CPU and increases contention. The availability read itself must also be protected; putting an unlocked read in the loop condition would introduce a data race.
- **Sleep and poll:** `usleep(100)` in a loop. This is better, but you either react late or poll often.
- **Condition variable:** *"Put me to sleep, and wake me when something about this dongle changes."* The thread blocks rather than repeatedly checking. A notification makes it eligible to resume, but CPU scheduling and mutex acquisition can delay its return.

A condition variable (`pthread_cond_t`) is a **waiting room** tied to a **mutex** and a **condition you check yourself** (the *predicate*), for example *"the dongle is free, its cooldown is over, and I'm first in its queue."*

The subject: *"A condition variable (pthread_cond_t) may be used to manage waiting queues."*

`pthread_cond_init` / `pthread_cond_destroy`

```
int pthread_cond_init(pthread_cond_t *cond, const pthread_condattr_t *attr);
int pthread_cond_destroy(pthread_cond_t *cond);
```

- **What they do:** Initialize or destroy a condition variable. Use `attr = NULL`.
- **Returns:** `0`, or an error number (`init`: e.g. `EAGAIN`, `ENOMEM`).

- **Pitfall:** Destroying a condition variable while a thread is still blocked on it is **undefined behavior**. Make sure every thread has been woken and joined first.

`pthread_cond_wait`

```
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
```

- **What it does: Atomically unlocks `mutex` and puts the thread to sleep** on `cond`. When the thread is woken, it **locks `mutex` again** before returning. So you always hold the mutex both before *and* after the call.
- **Returns:** `0`, or an error number. It never returns `EINTR`. Calling it without holding `mutex` is undefined behavior for the default mutex type. ⚠️ verify in man page
- **In Codexion:** A coder waiting for its dongles.

Why does `wait` need a mutex?

The mutex prevents the **lost wakeup**. Without it, this can happen:

```
Coder (waiter)                      Neighbour (releaser)
checks: dongle taken -> must wait
                                    releases dongle
                                    broadcasts "dongle changed!"  <- nobody is sleeping yet
goes to sleep                       ...
sleeps forever
```

A broadcast is **not remembered**. If nobody is waiting at that moment, it is simply lost. With the mutex:

1. The waiter holds the mutex while it checks the predicate.
2. The releaser must take the same mutex to change the state.
3. `pthread_cond_wait` releases the mutex and goes to sleep **as one step**.

So the releaser cannot slip in between "I checked" and "I'm asleep". All concurrent waiters on a given condition variable must use the same associated mutex; using different mutexes for simultaneous waits is undefined behaviour.

Why it must sit in a `while` loop

```
pthread_mutex_lock(&m);
while (!condition_is_true)            /* NOT: if (!condition_is_true) */
  pthread_cond_wait(&cond, &m);
/* here: condition_is_true AND we hold m */
pthread_mutex_unlock(&m);
```

Waking up does **not** mean your condition is true. There are three reasons:

1. **Spurious wakeups.** POSIX explicitly allows `pthread_cond_wait` to return even though nobody signaled.
2. **Stolen wakeups.** Between the broadcast and the moment you get the mutex back, another thread may have run and taken the dongle.
3. **Broadcast wakes everyone.** Each waiter must check "is it *my* turn?". In Codexion that means checking whether you are at the head of the queue.

The `while` loop handles all three: if the condition still isn't true, the thread goes back to sleep.

`pthread_cond_broadcast` (and why not `signal`)

```
int pthread_cond_broadcast(pthread_cond_t *cond);
```

- **What it does:** Unblocks **all** threads currently waiting on `cond`. If no one is waiting, nothing happens.
- **Returns:** `0`, or an error number.
- **In Codexion:**
  - When a dongle is released.
  - When the queue head changes.
  - When the simulation stops, so every waiter wakes up, sees the stop flag and exits.

Signal vs. broadcast: In general pthreads, `pthread_cond_signal` wakes *at least one* waiter, and `pthread_cond_broadcast` wakes *all* of them. - `signal` can reduce unnecessary wakeups when any selected waiter can make progress. More specialized protocols can use separate condition variables for different waiters. - With a priority queue, the thread you need to wake is a *specific* one (the queue head), and `signal` might wake the wrong one.

`signal` isn't allowed in Codexion anyway, so use `broadcast`. Waking everyone can cause unnecessary contention (the "thundering herd"). The predicate loop prevents a woken thread from proceeding incorrectly, provided state access and notification are synchronized correctly; broadcasting alone does not establish correctness. With one condition variable per dongle and only its two neighbours waiting on it, the number of unnecessary wakeups per broadcast is limited.

> **Tip:** Broadcast **while holding** the mutex. POSIX also allows broadcasting without it, but holding it is simpler to reason about. Helgrind also flags broadcasting without the lock as "dubious".

`pthread_cond_timedwait`

```
int pthread_cond_timedwait(pthread_cond_t *cond, pthread_mutex_t *mutex,
                           const struct timespec *abstime);
```

- **What it does:** Like `wait`, with a timeout at `abstime`. Timeout does not guarantee that the function returns at that exact instant: it must reacquire the mutex and get CPU time.
  - `abstime` is an **absolute** point in time, not a duration.
  - It is measured on the condition variable's clock, which is `CLOCK_REALTIME` by default. You can't change that here, because `pthread_condattr_setclock` isn't allowed. Conveniently, `gettimeofday` measures the same real-time clock.
- **Returns:** `0` when woken, or `ETIMEDOUT` when `abstime` has passed. Either way it **re-locks the mutex** before returning. It returns `EINVAL` if `tv_nsec` is outside `[0, 999999999]`. ⚠️ verify in man page
- **In Codexion:**
  - **Cooldown:** a waiting coder knows exactly when a dongle becomes usable (`release_time + dongle_cooldown`). It can sleep until then instead of polling.
  - **Periodic rechecks:** a finite timeout can let a waiter recheck stop or resource state even without a notification. It cannot repair a deadlocked mutex, and wall-clock changes can affect the wait.
- **Pitfall:** Passing a *duration* (e.g. `{0, 5000000}` for 5 ms) instead of an absolute time. That timestamp is in 1970, so the wait times out immediately and you get a busy-loop.

Minimal example: a gate with a timeout

```
#include <pthread.h>
#include <errno.h>
#include <time.h>

typedef struct s_gate
{
  pthread_mutex_t mtx;
  pthread_cond_t  cond;
  int       open;
} t_gate;

void  gate_open(t_gate *g)
{
  pthread_mutex_lock(&g->mtx);
  g->open = 1;
  pthread_cond_broadcast(&g->cond);
  pthread_mutex_unlock(&g->mtx);
}

int gate_wait_until(t_gate *g, long abs_ms)
{
  struct timespec ts;
  int       ret;

  ts.tv_sec = abs_ms / 1000;
  ts.tv_nsec = (abs_ms % 1000) * 1000000;
  ret = 0;
  pthread_mutex_lock(&g->mtx);
  while (!g->open && ret == 0)
    ret = pthread_cond_timedwait(&g->cond, &g->mtx, &ts);
  if (ret != 0 && ret != ETIMEDOUT)
    ret = -1;
  else
    ret = g->open;
  pthread_mutex_unlock(&g->mtx);
  return (ret);
}
```

Initialize `mtx` and `cond` with default attributes and set `open = 0` before using this example. `abs_ms` is a nonnegative real-time timestamp in ms, from `now_ms()` in §4. The function returns 1 if it observes the gate open, 0 if it times out with the gate still closed, and -1 for another wait error. It assumes mutex operations succeed; production error handling must cover initialization and other failures too.

A gate like this can serve as a **start gate**: threads wait while main finishes setup, initializes the common start time and deadlines, and opens the gate. This publishes a common start but does not make all threads run simultaneously. If you need every thread to have reached the gate first, also track readiness under a lock. Think through how a creation failure releases threads already waiting.

---

4. Time in C

`gettimeofday`

```
#include <sys/time.h>
int gettimeofday(struct timeval *tv, void *tz);   /* pass NULL for tz */
```

- **What it does:** Fills `tv` with the current wall-clock time: seconds (`tv_sec`) and microseconds (`tv_usec`) since 1970-01-01.

- **Returns:** `0`, or `-1` with `errno` set.
- **Notes:**
  - It reads the **real-time clock**, which can jump if the system time is adjusted. That is rare during a test run, and the subject explicitly recommends `gettimeofday`:

    > *"For this project, real-time measurements using gettimeofday() are acceptable and recommended for simplicity."*

  - This is a legacy wall-clock interface, explicitly allowed by this subject. Its declaration and availability should be checked in your platform headers and manual.

```
#include <stddef.h>
#include <sys/time.h>

long  now_ms(void)
{
  struct timeval  tv;

  gettimeofday(&tv, NULL);
  return (tv.tv_sec * 1000L + tv.tv_usec / 1000);
}
```

(This illustration assumes a 64-bit `long` and a successful `gettimeofday` call. Check the target platform and handle failure in your implementation. A 32-bit `long` cannot hold present-day epoch milliseconds; deadline additions and unit conversions also need overflow-safe arithmetic.)

Timestamps relative to the simulation start

The log shows *milliseconds since the simulation start*, not since 1970:

```
printed_timestamp = now_ms() - start_ms
```

- Record `start_ms` **once**, just before the coders begin, and store it where every thread can read it.
- Initialize each coder's `last_compile_start` to `start_ms`. The subject counts burnout from *"the beginning of their last compile or the beginning of the simulation"*.
- If the threads start one by one over a few ms, a start barrier (like the gate above) makes "start" mean the same moment for everyone.

`usleep`

```
#include <unistd.h>
int usleep(useconds_t usec);
```

- **What it does:** Suspends the calling **thread** for at least `usec` microseconds on an uninterrupted successful call. A signal can interrupt the sleep early.
- **Returns:** `0`, or `-1` with `errno` set to `EINTR` (interrupted by a signal) or `EINVAL` (on some systems, `usec >= 1000000`). ⚠️ verify in man page
- **Note:** It was removed from POSIX.1-2008 but is still widely available. The subject allows it.

Why `usleep` is imprecise

The subject itself warns that timing isn't perfect:

> *"Precision requirement: Burnout logs must be displayed within 10 ms of the actual burnout time. Allow a minimal tolerance when testing, as hardware and OS scheduling may slightly affect measured timings."*

"OS scheduling" is the hub manager from §2.5.

An uninterrupted successful `usleep(200000)` waits at least 200 ms; it does not promise an exact 200 ms duration. Recheck the clock after interruptions or wakeups:

- **Wake-up latency.** After the timer fires, your thread must be **scheduled** again. With 200 threads on 8 cores it may wait in line.
- **Timer granularity and slack.** The kernel rounds and groups timers to save power. ⚠️ verify in man page (`man 7 time`, timer slack)
- **Delays affect later phases.** Logging, lock contention and oversleep extend the interval between compile starts. Since each compile start resets that coder's deadline, total drift from the original simulation start is not itself a burnout condition; the gap between successive compile starts matters.

Building a more accurate sleep

The trick has two parts: 1. Sleep until an **absolute target time**, computed once. 2. Sleep in **small chunks**, re-checking the clock (and the stop flag) after each chunk.

```
void  sleep_until(long target_ms, t_sim *sim)
{
  while (now_ms() < target_ms && !sim_is_stopped(sim))
    usleep(500);
}
```

Here `sim_is_stopped` is your own function, and it reads the stop flag **under its mutex**.

- **Chunk size is a trade-off.** Smaller chunks (e.g. 100–500 µs) allow more frequent checks but add wakeup overhead. Larger chunks react less frequently. Neither gives an exact scheduling guarantee; measure on your machine.
- **Checking the stop flag** means a debugging coder notices the end of the simulation quickly, instead of sleeping 500 more ms while main waits in `pthread_join`.
- **Use phase start times as targets.** For example, `compile_start + time_to_compile` instead of "now + time_to_compile". This avoids adding the setup delay before sleeping to that phase. Scheduling delays between phases and cycles can still accumulate; absolute targets do not eliminate all drift.

---

5. Concurrency problems

Data race

What it is: Conflicting accesses to the same memory from different threads, at least one a write and at least one non-atomic, without the required synchronization ordering. They need not execute at the exact same wall-clock instant; merely appearing to run one after the other is not synchronization (§2).

Codexion suspects: - `last_compile_start` (coder writes, monitor reads) - the compile count - the stop flag - a dongle's state - the waiting queue

A suitable approach here: protect shared mutable values with a mutex used consistently by readers and writers. Immutable configuration published before thread creation is different: it needs no per-read lock because no concurrent thread changes it.

Deadlock

What it is: A group of threads each waits for something another thread in the group holds. Nobody can move.

The classic Codexion deadlock: 1. All N coders grab their **left** dongle at the same moment. 2. Each one now waits for their **right** dongle. 3. The right dongle is the left dongle of their neighbour, so it never becomes free.

The four Coffman conditions. For the resource-allocation deadlock described here, all four are necessary. Breaking one prevents that resource deadlock, but does not prevent unrelated hangs such as a lost notification or waiting for a thread that will never exit.

Condition	Meaning	In Codexion
Mutual exclusion	A resource can be held by only one thread	A dongle is in one hand only. You can't change this; it's the rule.
Hold and wait	A thread holds one resource while waiting for another	Holding the left dongle while waiting for the right one
No preemption	You can't take a resource away from its holder	Coders don't snatch dongles from each other
Circular wait	A cycle: A waits for B, B waits for C, ..., someone waits for A	Coder 1 → 2 → ... → N → 1 around the table

How resource ordering breaks circular wait:

- Number the dongles. Every coder always takes the **lower-numbered** dongle first, then the higher one.
- For most coders that's "left, then right". But the last coder, between dongle N and dongle 1, takes **dongle 1 first**.
- Now the dependency arrows can't form a full circle. At least one coder competes for their *first* dongle instead of holding one and blocking, and the chain breaks.

```
Without ordering:            With ordering (lower id first):
C1 holds D1 wants D2         C1 holds D1 wants D2
C2 holds D2 wants D3         C2 holds D2 wants D3
C3 holds D3 wants D1  <-cycle C3 wants D1 first -> blocked on D1, holds nothing
                              -> C2 gets D3 eventually, cycle impossible
```

Breaking hold-and-wait instead: a coder takes **both dongles at once or none**. The coder waits (without holding anything) until both are free, past their cooldown, and it is their turn, then takes both in one step.

Which approach fits better depends on how you design your queues (§8). Either way, write down your choice: the README must explain it.

> A consistent global order is also a useful rule for **mutexes**. If a code path ever locks two mutexes, every path that locks both must lock them **in the same global order**.

Starvation

What it is: The system as a whole keeps making progress, but one thread **never** gets its turn.

Example: Coders 1 and 3 keep alternating on the dongles that coder 2 needs. Each time one finishes, the other is ready, and coder 2 always loses the race and burns out.

This is why the subject requires **fair arbitration** through a scheduler, instead of "whoever calls `lock` first wins". The mutex itself makes **no fairness promise** about which waiter gets it next.

Busy-waiting

What it is: Looping to check a condition without sleeping: `while (!ready) ;` or a loop that locks, checks and unlocks with no sleep.

Why it hurts: It wastes a whole CPU core. With many coders, the busy threads take CPU time from the threads that need to work, so timings slip and coders burn out. Valgrind runs threads one at a time, so busy-waiting makes it crawl.

Summary table

Problem	Symptom	Typical fix
Data race	Random wrong values, "impossible" logs, works on your machine but fails at evaluation, reports from Helgrind or TSan	Protect conflicting accesses to shared mutable state with the same mutex
Deadlock	Program freezes. Usually all coders log "has taken a dongle" and then nothing happens	Break a Coffman condition: lock ordering, or take both dongles or none
Starvation	One specific coder burns out even though the timings are feasible	Fair arbitration: FIFO or EDF priority queue
Busy-waiting	100% CPU, burnouts appear with many coders, Valgrind extremely slow	Condition variables, or `usleep` in small chunks
Lost wakeup	A coder waits forever even though the dongle is free	Check the predicate under the mutex; `while` + `cond_wait`; use `timedwait` as a safety net
Hang at exit	Program prints the burnout but never returns	Broadcast every condition variable on stop; all loops check the stop flag

---

6. Scheduling: FIFO vs. EDF

When several coders want the same dongle, **the scheduler decides** who gets it. In the subject's words:

- **`fifo`:** *"the dongle is granted to the coder whose request arrived first."*
- **`edf`:** Earliest Deadline First, with *"deadline = last_compile_start + time_to_burnout"*. The coder closest to burning out goes first.

The subject also requires **liveness**: *"coders should not starve under edf scheduling, provided the parameters are feasible."*

Worked timeline (one contested dongle, simplified)

Setup (a snapshot of pending requests, not a full simulation trace): - `time_to_burnout = 800`, `time_to_compile = 200`, `dongle_cooldown = 100`. - Only one dongle is shown. In reality a coder needs two, but one is enough to see the difference.

Coder	Last compile start	Deadline (EDF key)	Requests the dongle at
A	0	0 + 800 = **800**	580
B	300	300 + 800 = **1100**	550

The dongle becomes available at **t = 600**. These request times can include waiting for another resource before requesting the dongle shown.

With FIFO. B's request arrived first (550 < 580), so B goes first.

```
t=600  B takes it, compiles until 800, releases
t=800  A reaches its deadline while the dongle is still unavailable
t=900  dongle would become usable again (800 + 100 cooldown), too late for A
```

With EDF. A's deadline is earlier (800 < 1100), so A goes first.

```
t=600  A takes it, compiles until 800, releases       (A starts before 800)
t=900  dongle usable again
t=900  B takes it                                    (B starts before 1100)
```

What this shows: - **FIFO** orders competing requests by arrival; it does not account for urgency. - **EDF** orders competing requests by deadline. A coder's deadline moves forward when they start compiling, while a waiting coder's deadline stays fixed. This gives waiting coders increasing urgency relative to coders who have recently compiled.

This example assumes each selected coder can acquire their other dongle too. It explains the priority rule, but it does not prove that the full simulation meets every deadline. A request can have the highest priority and still be blocked by a resource in use or in cooldown. Deadlock prevention, eventual progress, and starting before burnout are separate properties: check each one in your design.

The subject requires liveness under EDF with feasible parameters, but does not define a feasibility test. Distinguish a set of timings for which a successful schedule exists from one your particular implementation can sustain. Use hand-worked timelines and repeated tests to examine the difference; successful runs provide evidence, not a general proof.

Ties (equal deadlines in EDF) are not specified by the subject. Pick a deterministic rule, such as earliest request first or lowest coder id, and document it.

The subject also doesn't say exactly *when* a request "arrives". A natural choice is the protected registration of a resource request after refactoring. If requests are made separately for each dongle, distinguish the arrival of each request from the start of the whole acquisition attempt. Define this consistently, then document it.

---

7. Priority queues and binary min-heaps

Why the project needs one

For these policies, use a **min-priority queue**: it stores items and returns the one with the **smallest key** first. Priority queues can also be designed to return the largest key. Both policies fit this model:

Scheduler	Key
`edf`	deadline (`last_compile_start + time_to_burnout`)

Scheduler	Key
`fifo`	arrival order: a ticket number from an increasing counter

So **one heap implementation**, with a configurable comparison, can serve both policies. This does not require a single shared queue; queue placement is a separate design choice.

The subject requires it: *"You must implement a priority queue (heap) for FIFO/EDF scheduling."*

Nice property: While a coder waits, their deadline doesn't change, because it only changes when they *start* compiling. So keys never change while an item is in the heap, and you don't need a "decrease-key" operation.

For deterministic ordering, use a pair as the key: `(deadline, ticket)` for EDF and `(ticket)` for FIFO. A heap is **not stable**: equal keys come out in no guaranteed order, so the ticket breaks ties.

Array representation

A binary min-heap is a complete binary tree where **every parent ≤ its children**. It can be stored in a plain array without child pointers:

```
parent(i) = (i - 1) / 2
left(i)   = 2 * i + 1
right(i)  = 2 * i + 2
```

```
array: [ 3 | 5 | 8 | 9 | 6 | 12 ]
index:   0   1   2   3   4   5

               3 (0)
             /       \
        5 (1)          8 (2)
       /     \        /
    9 (3)   6 (4)  12 (5)
```

The minimum is always at index 0, so peeking is O(1).

Sift-up (used by insert)

Put the new item at the end, then swap it with its parent while it is smaller. Example: insert `4` (the root has no parent; only use the parent formula for `i > 0`).

```
append at index 6:   [3, 5, 8, 9, 6, 12, 4]     parent(6)=2 -> 8 > 4, swap
                     [3, 5, 4, 9, 6, 12, 8]     parent(2)=0 -> 3 < 4, stop
```

Sift-down (used by pop)

Take the root, move the **last** item to the root, then swap it with its **smaller** child while it is bigger. Example: pop from `[3, 5, 4, 9, 6, 12, 8]`.

```
remove 3, move 8 to root: [8, 5, 4, 9, 6, 12]   children 5,4 -> smaller is 4 (idx 2), swap
                          [4, 5, 8, 9, 6, 12]   children of idx 2: idx 5 (12) -> 8 < 12, stop
```

Insert and pop are both **O(log n)**, because the tree height is log n.

Illustration: sift-up on plain ints

```
static void swap_int(int *a, int *b)
{
  int tmp;

  tmp = *a;
  *a = *b;
  *b = tmp;
}

void  sift_up(int *heap, int i)
{
  int parent;

  while (i > 0)
  {
    parent = (i - 1) / 2;
    if (heap[parent] <= heap[i])
      return ;
    swap_int(&heap[parent], &heap[i]);
    i = parent;
  }
}
```

Your turn: - Write `sift_down` yourself. - Generalize the heap from `int` to your own request struct, with a comparison that depends on the scheduler. - Test the heap **alone, without threads**: insert random keys, pop them all, and check they come out sorted.

Capacity: If each coder has at most one outstanding request per queue, the maximum is 2 for a queue per dongle (its two neighbours), or N for one global queue. Prevent duplicate insertions; a single-coder table has only one possible requester. So you can allocate once at startup and never `realloc` while threads are running.

Thread safety: The heap itself has no lock. Protect it with the mutex of whatever owns it, which is the same mutex its condition variable uses.

---

8. Architecture overview

This section has no code on purpose: these are the design decisions you have to make.

Threads

One possible division of responsibilities (initialization failure also needs a stop-and-join path):

```
                main thread
     parse args -> init everything -> start threads
                      |                      |
        +-------------+---------+            |
        v               v         v          v
    coder 1  ...    coder N           monitor thread
   (loop:           (loop)            (loop every ~1 ms:
    request ->                         anyone past deadline?
    compile ->                         everyone done?
    debug ->                           -> set stop, wake all)
    refactor)
       |               |                      |
       +---------------+----------------------+
                       v
          main: join all -> destroy -> free
```

- **Main thread:** parses and validates arguments, allocates and initializes everything, starts the threads, joins them, and cleans up.
- **N coder threads:** the compile → debug → refactor cycle.
- **1 monitor thread:** required by the subject. It detects burnout (and logs it within 10 ms) and checks whether every coder has reached `number_of_compiles_required`.

Shared data → who touches it → which lock

Shared data	Written by	Read by	Protected by (one possible design)
Stop flag	monitor; main on startup failure	everyone	a "simulation state" mutex (or the print mutex, see §9)
`last_compile_start` of coder i	coder i	monitor, EDF scheduler	a per-coder mutex
Compile count of coder i	coder i	monitor	the same per-coder mutex
Dongle state (held? `available_at` = release time + cooldown)	coders	coders	that dongle's mutex
Waiting queue (heap)	coders	coders	the mutex the queue's condition variable uses
stdout	everyone	—	print mutex

Design questions to answer before you code

1. **Where are the queues?**
   - The subject says the scheduler is *"used by dongles"*, which suggests one queue per dongle.
   - A coder needs two dongles, so they may be in two queues at once. Will they acquire dongles separately in a fixed order, or reserve both together? How will the chosen protocol prevent deadlock?
   - Would one central arbiter be simpler? What are the trade-offs?
2. **Which Coffman condition do you break?** Lock ordering, all-or-nothing, or both?
3. **What exactly does a waiting coder wait for?** Write the predicate for your chosen protocol, e.g. *"the next dongle I need is free, its cooldown has expired, and my request has priority; or the simulation has stopped."* A protocol that reserves both dongles needs a predicate covering both.

4. **How does a waiting coder handle the cooldown?** A `pthread_cond_timedwait` until `available_at` makes that expiry a reason to recheck; scheduling and mutex contention may delay the return.
5. **What happens with 1 coder?** The subject says there is then only one dongle on the table. Compiling needs two, so the coder can **never** compile and must burn out at `time_to_burnout`. Don't hand them the same dongle twice. The subject doesn't say what the log should look like. Whatever you do, the coder thread must still **exit** so it can be joined.
6. **How do you validate arguments?**
    - `atoi` does not provide useful error reporting. Validate the full token and check the range **before** each arithmetic step in a custom parser. Also check allocation-size multiplication and deadline/unit-conversion arithmetic; accepting a number safely does not make every later calculation safe.
    - The subject asks you to reject negatives, non-integers and unknown schedulers. `number_of_coders = 0` should be rejected: the Overview specifies "one or more coders". Whether `0` is valid for the *other* arguments (e.g. `dongle_cooldown = 0`, `number_of_compiles_required = 0`) is not specified, so decide and document it.

Choosing a resource-acquisition protocol

The subject requires two dongles during compiling, but your design decides how a coder acquires them. Two useful starting points are:

Approach	How it works	What you must reason about
Acquire separately in a fixed order	Every coder requests the lower-numbered dongle first, then the higher-numbered one. They may hold the first while waiting for the second.	The resource order prevents circular wait, but holding an unused dongle can delay neighbours. Apply FIFO/EDF to each grant and test the resulting waiting times.
Reserve both together	A coder registers a request and waits until both dongles can be granted. Checking and reserving them is one protected operation.	This avoids holding one dongle while waiting, but queue priorities can leave resources idle. Establish a consistent order across queues and explain why some request can progress.

Neither approach automatically guarantees that deadlines are met. I recommend first writing down the state transitions and lock order for the simpler protocol you can explain, then working through a small timeline before implementing it. Keep request comparison separate from resource acquisition so FIFO and EDF can use the same synchronization code.

Also distinguish **owning a dongle** from **holding its state mutex**. A coder can mark a dongle as owned, release the mutex, and use the dongle while compiling. Other threads can then lock the mutex briefly to inspect its state. Holding a state mutex throughout compiling would prevent those inspections and complicate waiting and shutdown.

For each request, define when it is inserted, when it is removed, and what happens if the simulation stops before it is granted. Read the EDF deadline under the coder's state lock and copy it into the request; it remains fixed while the coder waits. Assign FIFO tickets under a shared lock at a documented arrival point. If a request participates in multiple queues, consistent comparison keys avoid contradictory priority orders between those queues.

Implementing waiting without spinning

A condition variable is a notification mechanism; the predicate is the actual rule for proceeding. For each waiting point:

1. Lock the mutex protecting the state you will inspect.
2. Check stop, ownership, cooldown, and request priority as applicable.
3. If the request can be granted, update ownership and queue state while still protected.
4. Otherwise, wait and check again after waking. Waiting releases its associated mutex and reacquires it before returning.

A condition wait releases only the mutex passed to it. If your predicate uses two dongle mutexes, you must arrange to release the other before sleeping and reacquire locks in the documented order before checking the full predicate again. Never sleep while keeping a mutex that another thread needs to make the predicate true.

Choose notifications to match the predicate. Waiting for one dongle can use that dongle's condition variable. A predicate involving multiple dongles needs a plan for changes to either resource: for example, a shared scheduler condition with its own consistently protected state, or per-dongle notifications combined with bounded timed rechecks. A shared scheduler lock can simplify coordination, but you must still protect each dongle's state as required by the subject and define the order between these locks.

Cooldown expiry is different from release: time passes without a thread changing state or broadcasting. Use an absolute timed wait for a relevant future cooldown expiry, then recheck. If progress also depends on another resource or shutdown, notifications must cover those events too. A bounded timeout can provide a fallback, at the cost of extra wakeups. Avoid repeatedly waiting on an expiry that is already in the past, which makes the loop spin.

On release, update ownership and cooldown under the protecting mutex before notifying waiters. On stop, wake all relevant waiters. After every wakeup, recheck the predicate: a broadcast neither reserves a dongle nor establishes which request gets it. If stop uses a different mutex, checking stop and then sleeping must still be coordinated with the stop notification; otherwise a stop broadcast can be missed between those operations.

Questions to investigate with a small experiment

Before choosing a protocol, draw a timeline for three or four coders. Which dongles can be used at the same time? What changes when you add cooldown? If a high-priority request is blocked, does some other request still make progress?

For each shared-state operation, ask: who reads this value, who writes it, and which mutex orders those accesses? For each wait, ask: what must become true, who can make it true, and what will wake the waiter? Try changing the thread start order and see whether your explanation still holds.

Use these questions alongside the subject and pthread documentation. The examples above are starting points for a design you can explain and test yourself.

The monitor thread

Its loop: 1. For each coder, read `last_compile_start` under that coder's lock and compare it with `now`. 2. If a coder is past its deadline, set stop, log `burned out`, and wake everyone. 3. If every coder has reached the required number of compiles, set stop and wake everyone. 4. Sleep a little (e.g. 0.5–1 ms) and repeat.

The sleep matters. With no sleep, this is busy-waiting. With a long sleep, you can miss the 10 ms window. A short polling interval helps responsiveness but does not guarantee a maximum latency: the OS must schedule the monitor, and locks and output can delay it too. Measure the full decision-to-log path.

Whether the deadline check uses > or >= is not specified by the subject. Pick one and be consistent.

Make the burnout decision consistent with compile starts. Locking a read prevents a data race, but a stale snapshot can still lead to an incorrect decision. For example, with a 100 ms burnout interval, the monitor reads a last-start value of 0 at time 80, then pauses. The coder starts again at 90. If the monitor resumes at 110 and compares its old snapshot with the new time, it may incorrectly announce burnout.

Choose a synchronization protocol in which checking expiry and committing stop are ordered against starting a compile. For an example to reason through, consider a simulation-state mutex shared by both operations: the coder checks stop and their current deadline before recording a compile start; the monitor checks the current deadline and commits stop under that same mutex. If coder state also has a separate mutex, both paths must take the locks in the same documented order. A coder whose deadline has already expired must not reset it and erase a missed deadline while waiting for the monitor to run.

Coordinate that decision with the print gate in §9. For example, the monitor can acquire the print mutex, then the simulation-state mutex, validate the deadline, and commit stop before printing the burnout line. Every path taking those locks must follow that order. Release state locks before broadcasting on resource condition variables, following your resource-lock order. Do not acquire the print mutex while already holding the simulation-state mutex in another path.

Define compile-count updates too: the stopping condition says coders have compiled, so distinguish starting a compile from completing its full phase. Where would you update the count, and how would you handle stop during that phase? The monitor's completion decision should use protected counts and a documented rule for a completion and burnout that occur together. Reaching one coder's individual target does not, by itself, satisfy the global stopping condition; think carefully before making that coder exit early.

Clean shutdown checklist

- ☐ Stop transition made under its protecting lock; repeated stop requests are harmless
- ☐ Every waiter has a reliable exit path: stop notification synchronized with its predicate, or a bounded timed recheck. Broadcasting each relevant condition variable while holding its associated mutex is one straightforward approach
- ☐ Every loop (waiting, sleeping, the monitor) checks the stop flag
- ☐ A coder that exits while holding dongles releases them, or at least unlocks every mutex it holds
- ☐ Main joins **all** threads, including the ones created before any `pthread_create` failure
- ☐ Only then: destroy all mutexes and condition variables, then free all memory
- ☐ All project allocations are freed; investigate lost and still-reachable allocations rather than assuming every tool/runtime allocation belongs to your code

---

9. Logging

Format (exactly as in the subject)

```
timestamp_in_ms X has taken a dongle
timestamp_in_ms X is compiling
timestamp_in_ms X is debugging
timestamp_in_ms X is refactoring
timestamp_in_ms X burned out
```

- `X` is the coder number. Coders are numbered from **1** to `number_of_coders`, not from 0.
- `timestamp_in_ms` is ms since the simulation start.
- Before each `is compiling` there are two `has taken a dongle` lines **for that coder**, one per dongle. Other coders' lines can appear between them.

Why output needs a mutex

The subject: *"Logging must be serialized so that two messages never interleave on a single line (use a mutex to protect output)."*

- **`write`** writes directly to the file descriptor, with no buffering. Two threads that each build a line with several `write` calls (number, space, text...) can interleave their pieces.
- **`printf`.** POSIX requires stdio functions to lock the `FILE` internally, so one `printf` call won't tear in the middle. ⚠️ verify in man page (`man 3p flockfile`). Even so, you still need your own mutex, for two reasons:
  1. **Order and consistency.** You read the time, then print. Without a lock, another thread can print between your time read and your print, and timestamps appear out of order.
  2. **Nothing may be printed after a burnout** (see below). One way to enforce that is to **check the stop flag and print inside the same critical section**, coordinated with the monitor's stop decision.
- **Don't mix** `write(1, …)` and `printf` on stdout. `printf` buffers and `write` doesn't, so the output can come out in the wrong order. When stdout is a pipe (e.g. `./codexion … | grep`), `printf` is fully buffered.

Nothing after a burnout

The subject says the simulation stops at burnout, and its examples end with the burnout line. This guide interprets that as requiring the burnout to be the **last** state message; the subject does not separately spell out how to suppress in-flight log attempts. Here is the race to avoid:

```
monitor: "coder 3 burned out" printed
coder 5: (already past its stop check) prints "is debugging"   <- wrong
```

The idea: have a single "print gate". The print function locks the print mutex, checks whether the simulation has stopped, prints only if it hasn't, and unlocks. The monitor announces a burnout **inside that same lock**: it sets stop and prints the burnout line. After that, every other print sees stop and stays silent. If the stop flag lives behind a different mutex, always take the two locks in the same order.

The 10 ms rule means the print critical section should be short. Sleeping while holding it delays every logger, and even output itself can block on a slow reader. A mutex ensures serialization; it does not guarantee bounded output latency. With `printf`, buffering also matters: a line timestamp does not prove when a reader actually received the line.

Error messages: using `stderr` with `fprintf(stderr, …)` is a useful convention. The subject does not prescribe the exact diagnostic text, stream or exit status.

---

10. Testing and debugging

Edge cases to try

> **Ready-made checker:** this repo has a basic one in `tests/`. `tests/run_tests.sh` opens a terminal selector; `tests/run_tests.sh --batch` runs baseline cases without interaction. The suite includes many of the scenarios below and checks observable log behaviour. It cannot prove request ordering or exact resource-release timing; see its README for assumptions and limits. Add your own cases to `tests/cases.txt`.

The expected outcomes below are this guide's own reasoning; the subject doesn't give official test cases. Derive them yourself on paper before you trust them.

Command	Why it's interesting	Expected
`./codexion 1 800 200 200 200 5 0 fifo`	Only one dongle exists	Can never compile, so it burns out at ~800. The program must still **exit cleanly**
`./codexion 2 800 200 200 200 5 0 fifo`	Smallest real contention; the two coders share the same two dongles	Runs without burnout until both have compiled 5 times
`./codexion 2 800 200 100 100 5 500 fifo`	Long cooldown	Assuming coder 1 starts at 0, coder 2 can start at ~700. Coder 1 cannot restart before its deadline at ~800. Either coder may win the initial grant
`./codexion 4 410 200 100 100 10 0 edf`	Tight but feasible: pairs alternate exactly	A successful timeline exists with little margin; use as an experiment, not a universal scheduler expectation
`./codexion 4 310 200 100 100 10 0 edf`	Infeasible	A burnout at ~310
`./codexion 200 800 200 200 200 5 0 edf`	Many threads	No burnout, reasonable CPU, clean exit
`./codexion 5 800 200 200 200 0 0 fifo`	0 required compiles	Not specified by the subject. Decide and document
`./codexion 0 800 200 200 200 5 0 fifo`	Zero coders	Rejected as invalid input. No threads started
Invalid: `-5`, `abc`, `12x`, `FIFO`, wrong argument count	Validation	Rejected, no simulation started, no leaks
Very large integers, e.g. `99999999999`	Numeric range and allocation checks	Reject values outside your documented supported range without overflow; the subject does not impose `INT_MAX` as a universal limit

Also check: - [ ] **Run each test many times.** Concurrency bugs are random, so a loop that runs 100 times and greps for problems is worth writing. - [ ] **Watch the CPU.** Run `top` or `htop` while the program runs. High CPU with nonzero phases is a reason to inspect spinning, frequent wakeups and logging; it does not identify the cause by itself. - [ ] **Check the 10 ms burnout window.** Compare the burnout timestamp with `last_compile_start + time_to_burnout` (you can log the latter temporarily while debugging). - [ ] **Check the log rules.** Under a stable wall clock, timestamps should not go backwards; each `is compiling` follows two takes by the same coder; nothing appears after `burned out`. Distinguish printed event timestamps from when output actually reaches a reader.

Race detectors

Valgrind's thread checkers. Both find races and lock problems, and Helgrind also reports inconsistent lock ordering (potential deadlocks):

```
valgrind --tool=helgrind ./codexion 4 800 200 200 200 3 0 fifo
valgrind --tool=drd      ./codexion 4 800 200 200 200 3 0 fifo
```

ThreadSanitizer is built into the compiler and is much faster than Valgrind:

```
cc -Wall -Wextra -Werror -pthread -g -fsanitize=thread *.c -o codexion_tsan
./codexion_tsan 4 800 200 200 200 3 0 fifo
```

Rules: - **Never combine `-fsanitize=thread` with Valgrind.** They conflict. Run them separately. TSan also can't be combined with `-fsanitize=address`. - On some newer Linux kernels, TSan fails at startup with *"unexpected memory mapping"*. A local diagnostic is `setarch $(uname -m) -R ./codexion_tsan …`, which disables address randomization for that process. It may be restricted or ineffective; check compiler/runtime compatibility and do not change system-wide settings just to make a test pass. - Valgrind runs threads **one at a time** and much slower, so use **generous** timings (e.g. `time_to_burnout` 2000+). A burnout under Valgrind with tight timings does not establish a logic error. A race report needs investigation: confirm the accesses, synchronization, tool version and any suppression before deciding whether it reflects your code. - A separate sanitizer build is useful for debugging. The subject requires its compiler flags but does not explicitly ban a debug target; keep the normal build usable for evaluation.

Leak checks

```
valgrind --leak-check=full --show-leak-kinds=all ./codexion 4 800 200 200 200 3 0 fifo
```

Test leaks on **every exit path**: normal end, burnout, and invalid arguments. If `pthread_create` fails, handle it too. You can't easily force that failure, so review that path by reading the code.

How to read a race report

A Helgrind report (shortened and illustrative):

```
Possible data race during read of size 8 at 0x4A8B0D8 by thread #3
Locks held: none
   at 0x109A2F: monitor_loop (monitor.c:17)

This conflicts with a previous write of size 8 by thread #2
Locks held: 1, at address 0x4A8B040
   at 0x1094C1: coder_loop (coder.c:42)
```

A TSan report has the same structure:

```
WARNING: ThreadSanitizer: data race
  Read of size 8 at 0x7b0c00000010 by thread T3:
    #0 monitor_loop monitor.c:17
  Previous write of size 8 at 0x7b0c00000010 by thread T2:
    #0 coder_loop coder.c:42
```

How to read either one: 1. **Two accesses, same address.** Look up the two `file:line` pairs. Here: `monitor.c:17` reads and `coder.c:42` writes the same variable, probably `last_compile_start`. 2. **Check the locks.** One side holds a lock and the other holds none. Both sides must hold **the same** mutex. 3. **Fix the unprotected side**, not just the side the report lists first. 4. **Re-run several times.** Detectors only see races on paths that actually ran.

Compile with `-g` so you get file and line numbers.

---

11. Common mistakes checklist

Threads - [ ] Passing `&i` (the loop counter) to `pthread_create` - [ ] Not joining every thread, or joining the same thread twice - [ ] Not handling a failed `pthread_create`: the threads already started are left running

Mutexes - [ ] Reading shared mutable data without synchronization ("it's only a read") - [ ] An early `return` or `break` that skips an unlock - [ ] Locking two mutexes in different orders on different code paths - [ ] Sleeping or printing for a long time while holding a lock - [ ] Destroying mutexes or condition variables before all threads are joined

Condition variables - [ ] `if` instead of `while` around `pthread_cond_wait` - [ ] Changing the predicate's data
without holding the condition variable's mutex - [ ] Passing a duration instead of an absolute time to `pthread_cond_timedwait`, or `tv_nsec >= 1000000000` - [ ] Not broadcasting on stop, so the program hangs at join

Timing - [ ] Plain `usleep(time_to_x * 1000)`: drift adds up and stop is noticed late - [ ] Printing epoch milliseconds instead of ms since the simulation start - [ ] Coders not initialized with `last_compile_start = start` - [ ] A monitor that sleeps too long (misses 10 ms) or not at all (busy-wait)

Subject requirements and the guide's stopping/logging interpretation - [ ] A global variable anywhere (including `static` file-level variables) - [ ] A function that isn't on the allowed list (`pthread_detach`, `pthread_cond_signal`, `clock_gettime`, `strtol`…) - [ ] Cooldown not applied, or applied to only one of the two dongles - [ ] Scheduler ignored: "whoever locks first wins" is not FIFO - [ ] Coder numbers starting at 0 - [ ] Messages printed after `burned out` - [ ] The 1-coder case hangs instead of burning out and exiting - [ ] Leaks on the error or burnout path - [ ] Omitting required flags `-Wall -Wextra -Werror -pthread`, or relinking without changes

---

12. README requirements

The subject requires a `README.md` at the **root of the repository**, written in **English**, with at least:

1. **The very first line**, in italics: *This project has been created as part of the 42 curriculum by \<login1>[, \<login2>[, ...]].*
2. **"Description":** what the project is, its goal, and a short overview.
3. **"Instructions":** how to compile, install and run.

4. **"Resources":** classic references (docs, articles, tutorials) **and a description of how AI was used**: for which tasks and which parts of the project.

Codexion adds two more sections:

1. **"Blocking cases handled":** every concurrency problem you addressed. The subject names some: deadlock prevention and Coffman's conditions, starvation prevention, cooldown handling, precise burnout detection, and log serialization.
2. **"Thread synchronization mechanisms":** the primitives you used (`pthread_mutex_t`, `pthread_cond_t`, any custom event mechanism) and how they coordinate access to dongles, logging and monitor state. The subject wants **examples of how races are prevented** and **how coders and the monitor communicate safely**.

Writing an honest "AI usage" section

The subject's AI rules: use AI only for content **you fully understand and can take responsibility for**, check everything, and get peer review. During evaluation you may be asked to change your code live, so you have to understand every line.

A good AI section is **specific**. It says which tool, which task, which files, and how you checked the output. Example structure:

```
### AI usage
- **Tool:** <name of the assistant>
- **Used for:**
  - Explaining pthread concepts (condition variables, spurious wakeups) — I then read the man pages to confirm.
  - Reviewing my README wording.
  - Suggesting edge-case test commands (section "Testing").
- **Not used for:** writing the scheduler, heap, coder or monitor code.
- **Verification:** ran Helgrind, DRD and TSan; discussed the design with <peer login>.
```

If you used this guide, say so. If AI *did* write some code, say **which** code and how you checked it. Be honest: unclear AI use that you can't explain at the defense will cost you far more than a plain, honest list.

---

13. Further resources

API references checked for this guide: condition waits (POSIX), mutex locking (POSIX), and usleep (Linux). These explain API behaviour; the subject still determines which functions you may use.

Man pages. On Linux, `man 3p` gives the POSIX version and `man 3` the glibc version.

```
man 3p pthread_create    man 3p pthread_join
man 3p pthread_mutex_lock  man 3p pthread_mutex_init
man 3p pthread_cond_wait   man 3p pthread_cond_timedwait
man 3p pthread_cond_broadcast
man 3  gettimeofday      man 3  usleep
man 7  pthreads          man 7  time
```

If `man 3p` is missing, install the POSIX man pages (e.g. the `manpages-posix-dev` package on Debian/Ubuntu) or read the POSIX spec online: https://pubs.opengroup.org/onlinepubs/9799919799/

Tutorials and books - **LLNL: POSIX Threads Programming:** https://hpc-tutorials.llnl.gov/posix/ A classic, compact walk through creation, mutexes and condition variables. - **Operating Systems: Three Easy Pieces** (Arpaci-Dusseau), free online at https://pages.cs.wisc.edu/~remzi/OSTEP/. Read the *Concurrency* part: - *Concurrency: An Introduction* - *Thread API* - *Locks* - *Lock-based Concurrent Data Structures* - *Condition Variables* - *Common Concurrency Problems* (deadlock and the Coffman conditions) - **Valgrind**

manual: the Helgrind and DRD chapters, https://valgrind.org/docs/manual/ - **ThreadSanitizer docs:** https://clang.llvm.org/docs/ThreadSanitizer.html - **The dining philosophers problem** (Dijkstra, 1965), the classic that Codexion is built on. Any OS textbook covers it. - **Binary heaps:** chapter 6 ("Heapsort") of *Introduction to Algorithms* (CLRS), or any data-structures course.

Good luck. Draw your design on paper before you write any code, and have a peer try to break it.